

به نام خدا

گزارش پروژه

سیستم کلاس هوشمند

تیم آردا ویو (Arda View):

محمد مهدی اسدی

امیرحسین سلیمانی

محمد امین عبداللهزاده

مهرگان علیدادی

استاد راهنما:

امیر خرسندی

دستیار استاد:

محمد امین کافی

نسخه 1.0

بهمن ماه ۱۴۰۴

فهرست

۱. معرفی مسئله.....	2
۱.۱. بیان مسئله.....	2
۱.۲. هدف پروژه.....	2
۲. طراحی سیستم.....	3
۲.۱. معماری کلی سیستم.....	3
۲.۲. اجزای سخت افزاری.....	5
۲.۳. اجزای نرم افزاری.....	6
۲-۳-۱. بخش هوش مصنوعی Gesture Recognition.....	7
۲-۳-۲. منطق کنترل و ماشین حالت State Machine.....	9
۲-۳-۳. سرور Backend.....	11
۳. رابط کاربری و لایه نمایش Frontend.....	13
۳-۱. نمایش سه بعدی کلاس با Three.js.....	13
۳-۲. نمایش داده های حسگرها با ECharts.....	14
۳-۳. معماری رویدادمحور در Frontend.....	16
۴. ساختار فایل های پروژه.....	17
۵. نتایج و تحلیل عملکرد سیستم.....	20
۵-۱. نتایج کمی.....	20
۵-۲. تحلیل نقاط قوت.....	21
۵-۳. محدودیت ها.....	21
۶. پیشنهادات توسعه آینده.....	22
۷. جمع بندی نهایی.....	22
منابع.....	23

۱. معرفی مسئله

۱.۱. بیان مسئله

با گسترش فناوری‌های نوین در حوزه اینترنت اشیا (IoT) و هوش مصنوعی، مفهوم «فضاهای هوشمند» به‌عنوان یکی از کاربردی‌ترین حوزه‌های مهندسی کامپیوتر و برق مطرح شده است. کلاس‌های درس به‌عنوان یکی از مهم‌ترین محیط‌های آموزشی، ظرفیت بالایی برای هوشمندسازی دارند. در بسیاری از محیط‌های آموزشی، کنترل تجهیزاتی نظیر روشنایی، تهویه، درب‌ها و پرده‌ها به‌صورت دستی انجام می‌شود. علاوه بر این، پایش شرایط محیطی مانند دما، رطوبت، کیفیت هوا، شدت نور و میزان صدا اغلب به‌صورت دقیق و بلادرنگ صورت نمی‌گیرد. این موضوع می‌تواند بر کیفیت فرآیند یادگیری، تمرکز دانشجویان و حتی سلامت افراد تأثیر منفی بگذارد.

از سوی دیگر، تعامل انسان با سیستم‌های کنترلی معمولاً مبتنی بر کلیدهای فیزیکی، ریموت یا اپلیکیشن‌های موبایلی است. این روش‌ها اگرچه کاربردی هستند، اما همچنان نیازمند تماس فیزیکی یا استفاده از واسطه‌های سنتی می‌باشند. در این پروژه تلاش شده است تا با بهره‌گیری از بینایی ماشین و تشخیص حرکات دست، یک روش تعاملی طبیعی‌تر، هوشمندتر و مدرن‌تر برای کنترل تجهیزات کلاس ارائه شود.

۱.۲. هدف پروژه

هدف اصلی این پروژه طراحی و پیاده‌سازی یک سیستم یکپارچه برای «کلاس هوشمند» است که چهار لایه اصلی را پوشش دهد:

۱. لایه حسگرها و سخت‌افزار برای جمع‌آوری داده‌های محیطی
۲. لایه پردازش هوشمند برای تشخیص حرکات دست با استفاده از بینایی ماشین
۳. لایه سرور و ارتباط بلادرنگ برای مدیریت داده‌ها و دستورات
۴. لایه رابط کاربری گرافیکی برای نمایش اطلاعات و کنترل سیستم

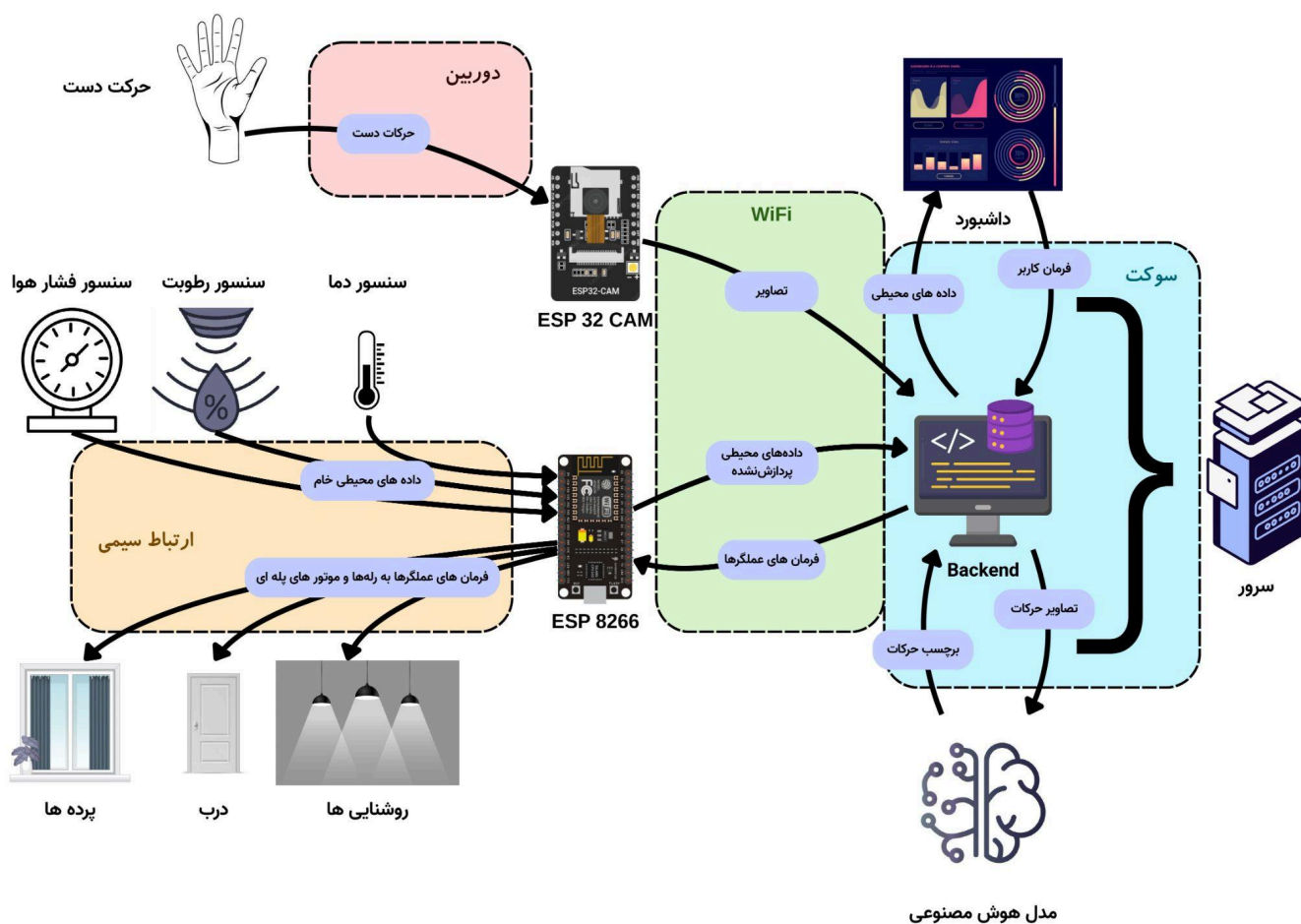
در این سیستم، کاربر می‌تواند تنها با استفاده از حرکات دست، تجهیزات کلاس را کنترل کند. همچنین تمامی پارامترهای محیطی به‌صورت لحظه‌ای جمع‌آوری شده و در یک داشبورد تحت وب نمایش داده می‌شوند.

۲. طراحی سیستم

۲.۱. معماری کلی سیستم

معماری این پروژه به صورت لایه‌ای و ماژولار طراحی شده است تا قابلیت توسعه و نگهداری آسان‌تری داشته باشد. ساختار کلی سیستم شامل سه بخش اصلی است:

- بخش سخت‌افزار (سنسورها و میکروکنترلرها)
- بخش پردازش و Backend
- بخش رابط کاربری (Frontend)



شکل ۱ - معماری کلی سیستم کلاس هوشمند

در این معماری، دوربین متصل به ماژول ESP32-CAM وظیفه دریافت تصویر از دست کاربر را بر عهده دارد. تصویر دریافت شده برای پردازش اولیه و تشخیص نقاط کلیدی دست استفاده می شود. خروجی این بخش، برچسب حرکت تشخیص داده شده است که به سرور ارسال می گردد.

در بخش دیگر، برد ESP8266 به مجموعه ای از سنسورها متصل است که شامل سنسور دما و رطوبت (BME280)، سنسور کیفیت هوا (MQ135)، سنسور نور (BH1750)، سنسور صدا و سنسور فشار هوا می باشد. این برد داده های محیطی را جمع آوری کرده و از طریق WebSocket به سرور ارسال می کند.

سرور مرکزی (پیاده سازی شده با FastAPI) نقش هسته مدیریتی سیستم را ایفا می کند. این سرور:

- داده های حسگرها را دریافت و ذخیره می کند
- دستورات Gesture را پردازش می کند
- وضعیت سیستم را مدیریت می کند (State Machine)
- اطلاعات را به داشبورد تحت وب ارسال می کند

در نهایت، رابط کاربری تحت وب که با Vite.js، Three.js و ECharts پیاده سازی شده است، داده ها را به صورت گرافیکی نمایش می دهد.

۲.۲. اجزای سخت‌افزاری

در این پروژه از ترکیبی از بردهای توسعه و سنسورهای محیطی استفاده شده است تا یک بستر کامل برای جمع‌آوری داده و اجرای دستورات فراهم شود.

توضیحات	نقش	قطعه
جمع‌آوری داده و دریافت دستور از سرور	واحد کنترل حسگرها و عملگرها	ESP8266
پردازش اولیه تصویر دست	دوربین برای تشخیص حرکات دست	ESP32-CAM
-	حسگر دما و رطوبت	BME280
تشخیص آلودگی هوا	حسگر مونوکسید کربن CO	MQ135
اندازه‌گیری شدت روشنایی	حسگر روشنایی	BH1750
-	حسگر فشار هوا	TM7711

ESP32-CAM

این ماژول به دلیل دارا بودن دوربین داخلی و توان پردازشی مناسب، برای دریافت تصویر دست کاربر انتخاب شده است. این برد از طریق WiFi به سرور متصل شده و داده‌های مرتبط با حرکت دست را ارسال می‌کند.

ESP8266

برد ESP8266 به عنوان گره جمع‌آوری داده‌های محیطی عمل می‌کند. این برد علاوه بر هزینه پایین و مصرف انرژی مناسب، قابلیت اتصال WiFi داخلی دارد و برای پروژه‌های IoT گزینه‌ای مناسب محسوب می‌شود.

سنسورها

در این پروژه از سنسورهای زیر استفاده شده است:

- BME280 برای اندازه‌گیری دما و رطوبت
- MQ135 برای پایش کیفیت هوا و CO2
- BH1750 برای اندازه‌گیری شدت نور
- سنسور صدا برای اندازه‌گیری سطح نویز محیط
- سنسور فشار هوا برای ثبت فشار محیط

این داده‌ها به صورت لحظه‌ای خوانده شده و به سرور ارسال می‌شوند تا در داشبورد نمایش داده شوند.

۲.۳. اجزای نرم‌افزاری

سیستم نرم‌افزاری پروژه شامل سه بخش اصلی است:

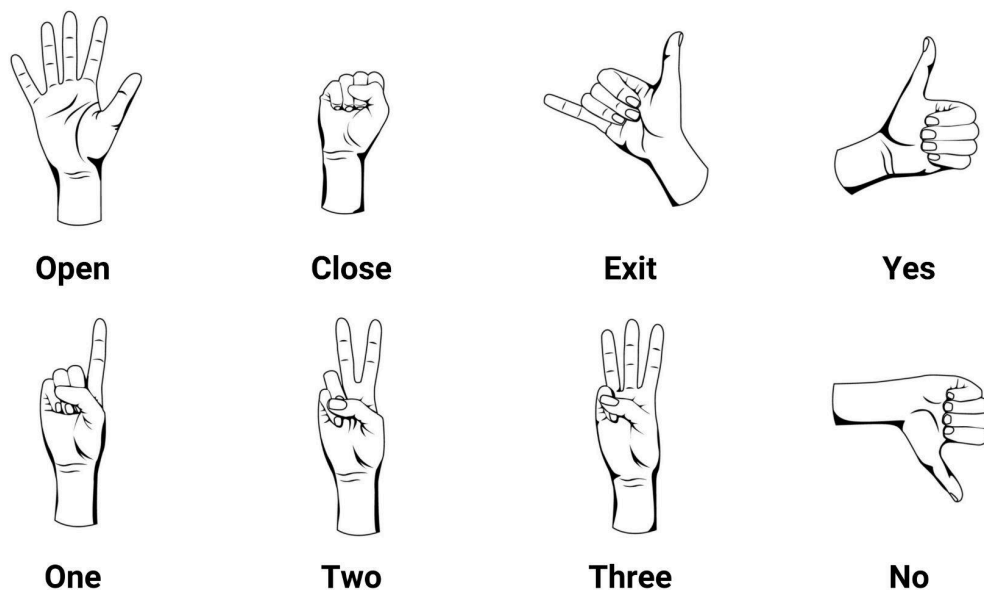
1. بخش هوش مصنوعی (تشخیص حرکات دست)
2. سرور Backend
3. رابط کاربری Frontend

۲-۳-۱. بخش هوش مصنوعی (Gesture Recognition)

در این پروژه برای تشخیص حرکات دست از کتابخانه MediaPipe استفاده شده است. این کتابخانه قابلیت تشخیص ۲۱ نقطه کلیدی دست را در هر فریم تصویر فراهم می‌کند. پس از استخراج مختصات این نقاط، داده‌ها به یک مدل یادگیری ماشین داده می‌شوند.

مدل‌های مورد استفاده شامل:

- Keypoint Classifier MLP برای تشخیص حالت‌های ثابت دست
- Point History Classifier LSTM برای تشخیص حرکات دینامیک



شکل ۲ - مجموعه حرکات دست تعریف‌شده در سیستم

این حرکات به گونه‌ای انتخاب شده‌اند که هم از نظر تشخیص توسط مدل دقت مناسبی داشته باشند و هم برای کاربر طبیعی و ساده باشند.

کد اصلی در app.py

استفاده از MediaPipe برای تشخیص 21 نقطه کلیدی دست

```
hands = mp_hands.Hands(  
    static_image_mode=False,  
    max_num_hands=1,  
    min_detection_confidence=0.7,  
    min_tracking_confidence=0.5,  
)
```

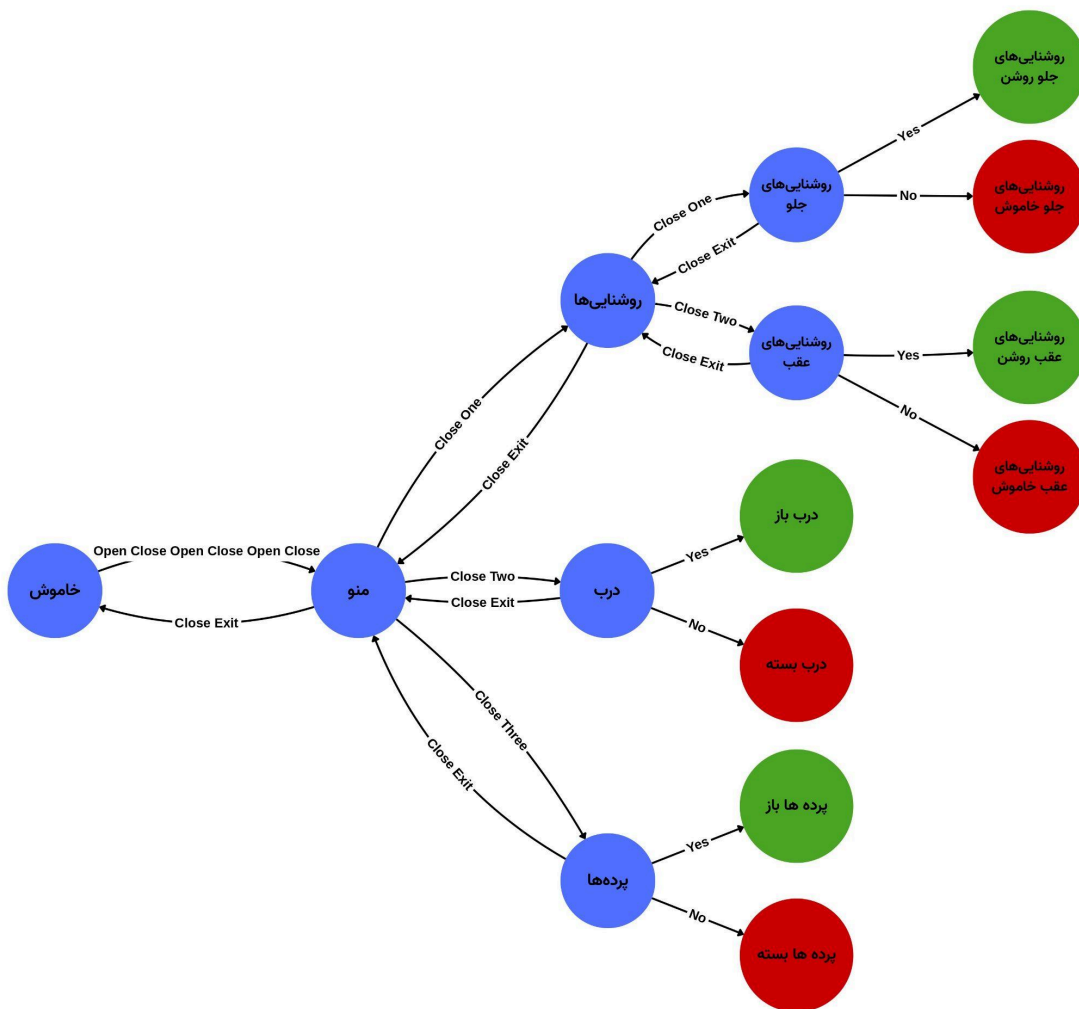
پردازش فریم و ارسال به مدل

```
results = hands.process(rgb_image)  
hand_sign_id = keypoint_classifier(landmarks)  
gesture_label = labels[hand_sign_id]
```

۲-۳-۲. منطق کنترل و ماشین حالت (State Machine)

برای جلوگیری از اجرای تصادفی دستورات، از یک ماشین حالت (State Machine) استفاده شده است. این طراحی باعث می‌شود سیستم تنها در صورت طی شدن یک توالی مشخص از حرکات، وارد حالت کنترل شود.

به‌عنوان مثال، برای خروج از حالت OFF باید توالی مشخصی از حرکات Open و Close اجرا شود. پس از آن، سیستم وارد حالت انتخاب (SELECT) می‌شود و کاربر می‌تواند نوع تجهیز (چراغ، درب، پرده) را انتخاب کند. سپس در مرحله ACTION، با حرکت Yes یا No فرمان نهایی صادر می‌شود.



شکل ۳ - نمودار ماشین حالت کنترل Gesture

حالت OFF: نیاز به توالی باز کردن قفل دارد

```
OFF_UNLOCK_SEQUENCE = ["Open", "Close",  
    "Open", "Close",  
    "Open", "Close"]
```

حالت SELECT: انتخاب دسته‌بندی

```
if gesture == "One":  
    state = LIGHTS # کنترل چراغ‌ها  
elif gesture == "Two":  
    state = DOOR # کنترل درب  
elif gesture == "Three":  
    state = CURTAIN # کنترل پرده
```

حالت ACTION: اجرای دستور

```
if state == LIGHTS and gesture == "Yes":  
    send_command("light", "on")
```

۲-۳-۳. سرور Backend

سرور پروژه با استفاده از FastAPI در زبان Python پیاده‌سازی شده است. این سرور از پروتکل WebSocket برای ارتباط بلادرنگ با کلاینت‌ها استفاده می‌کند. سه مسیر اصلی WebSocket در سیستم تعریف شده است:

- دریافت حرکات دست از ماژول AI
- دریافت داده‌های حسگرها از ESP8266
- ارسال داده‌ها به داشبورد

استفاده از WebSocket باعث شده است تا داده‌ها با تأخیر بسیار کم و به صورت Real-Time در اختیار رابط کاربری قرار گیرند.

اتصال به سرور

```
const ws = new WebSocket("ws://localhost:8000/ws/dashboard");
```

دریافت داده‌های سنسور

```
ws.onmessage = (event) => {
```

```
  const data = JSON.parse(event.data);
```

```
  if (data.type === "sensor") {
```

```
    emit("sensor:temperature", data.temperature);
```

```
    emit("sensor:humidity", data.humidity);
```

```
    // ...
```

```
  } else if (data.type === "action") {
```

```
emit("action:received", data);  
}  
};
```

۳. رابط کاربری و لایه نمایش (Frontend)

یکی از مهم‌ترین بخش‌های این پروژه، طراحی یک رابط کاربری مدرن، واکنش‌گرا و بلادرنگ است که بتواند داده‌های محیطی و وضعیت سیستم را به شکلی واضح و حرفه‌ای نمایش دهد. در این پروژه تلاش شده است تا علاوه بر نمایش داده‌های عددی، تجربه‌ای بصری و تعاملی برای کاربر فراهم شود.

رابط کاربری با استفاده از Vite.js و JavaScript پیاده‌سازی شده و برای بخش‌های گرافیکی از دو کتابخانه قدرتمند Three.js و Apache ECharts استفاده شده است.

۳-۱. نمایش سه‌بعدی کلاس با Three.js

در صفحه اصلی داشبورد، یک مدل سه‌بعدی از کلاس درس طراحی شده است که نمایی کلی از محیط را نمایش می‌دهد. این مدل سه‌بعدی شامل عناصر اصلی کلاس مانند دیوارها، درب، پنجره‌ها و تجهیزات داخلی است. هدف از این طراحی، ایجاد درک بصری بهتر از وضعیت سیستم و فراهم کردن تجربه‌ای مدرن‌تر نسبت به داشبوردهای سنتی بوده است.

در این نما، وضعیت اتصال ESP8266 و دوربین نیز به صورت گرافیکی نمایش داده می‌شود. همچنین هنگام اجرای یک دستور (مانند روشن شدن چراغ یا باز شدن پرده)، تغییرات مربوطه می‌تواند در مدل سه‌بعدی اعمال شود.



شکل ۴ - نمای اصلی داشبورد و مدل سه بعدی کلاس

۳-۲. نمایش داده‌های حسگرها با ECharts

برای نمایش داده‌های محیطی، از کتابخانه Apache ECharts استفاده شده است. این کتابخانه امکان رسم نمودارهای پویا و بلادرنگ را فراهم می‌کند.

در سیستم طراحی‌شده، برای هر سنسور یک نمودار خطی مستقل در نظر گرفته شده است. محور افقی (X) نمایانگر زمان و محور عمودی (Y) بیانگر مقدار سنسور است. داده‌ها به صورت پیوسته از طریق WebSocket دریافت شده و نمودارها به روزرسانی می‌شوند.

ویژگی‌های بخش نمودارها شامل موارد زیر است:

- نمایش هم‌زمان ۶ پارامتر محیطی
- به روزرسانی بلادرنگ بدون Refresh صفحه
- نمایش ۶۰ نمونه آخر برای جلوگیری از شلوغی نمودار
- طراحی ساده و خوانا برای تحلیل سریع داده‌ها



شکل ۵ - صفحه نمودارهای بلادرنک حسگرها

```
const chart = echarts.init(element);
```

```
chart.setOption({
```

```
  title: { text: "Temperature (°C)" },
```

```
  xAxis: { type: "category", data: timestamps },
```

```
  yAxis: { type: "value" },
```

```
  series: [{
```

```
    name: "Temp",
```

```
    type: "line",
```



```

    data: values,
    smooth: true
  ]]);

```

// بروزرسانی لحظه‌ای

```

on("sensor:temperature", (value) => {
  timestamps.push(new Date().toLocaleTimeString());
  values.push(value);
  chart.setOption({ ... });
});

```

۳-۳. معماری رویدادمحور در Frontend

در طراحی رابط کاربری از یک معماری Event-Driven استفاده شده است. در این معماری، هر زمان داده‌ای از سرور دریافت می‌شود، یک رویداد (Event) در سیستم منتشر شده و بخش‌های مرتبط به آن واکنش نشان می‌دهند.

به عنوان مثال:

- دریافت داده جدید دما → انتشار رویداد sensor:temperature → به‌روزرسانی نمودار مربوطه
- دریافت دستور جدید → انتشار رویداد action:received → ثبت در لاگ سیستم

این رویکرد باعث جداسازی مناسب ماژول‌ها، خوانایی بهتر کد و قابلیت توسعه ساده‌تر سیستم شده است.

۴. ساختار فایل‌های پروژه

```
smart-class-project/
├── ai-model/
│   ├── hand-gesture-recognition-mediapipe/
│   │   ├── app.py          برنامه اصلی
│   │   ├── model/
│   │   │   ├── keypoint_classifier/  مدل تشخیص حالت دست
│   │   │   └── point_history_classifier/  مدل حرکت انگشت
│   │   └── utils/
│   │       └── cvfpscalc.py  محاسبه FPS
│   └── hardware/
│       ├── test-esp32-cam/
│       │   ├── Blink/Blink.ino  تست LED
│       │   └── esp32-cam-blink/  تست ESP32-CAM
│       └── web-server/
│           └── backend/
```

```

|   └─ routes/

|   └─ gesture.py      # WebSocket حرکات دست
|   └─ esp8266.py      # WebSocket سنسورها
|
└─ frontend/_threejs/

    └─ src/

        └─ js/

            └─ main.js      # نقطه ورود

            └─ core/

                └─ events.js    # Event Bus

                └─ state.js     # State Management

                └─ backendSocket.js # ارتباط با سرور

                └─ ui/

                    └─ statusBar.js    # نوار وضعیت

                    └─ sensorPanel.js  # پنل سنسورها

                    └─ gesture.js     # نمایش حرکات

                    └─ charts/

                        └─ temperatureChart.js

```

| | | |—humidityChart.js

| | | |—... # نمودار دیگر

| | |—three/

| | |—classroom.js # شبیه ساز سه بعدی کلاس

| |—css/

| |—style.css

|—index.html

پروژه به صورت ماژولار و ساختاریافته سازمان دهی شده است تا هر بخش مسئولیت مشخصی داشته باشد. ساختار کلی شامل سه پوشه اصلی است:

1. /ai-model

شامل کدهای مربوط به پردازش تصویر و مدل های یادگیری ماشین

2. /hardware

شامل فایل های مربوط به تست و راه اندازی بردهای ESP

3. /web-server

شامل Frontend و Backend پروژه

این ساختار باعث شده است توسعه هر بخش به صورت مستقل امکان پذیر باشد و تیم های مختلف بتوانند روی بخش های جداگانه کار کنند.

۵. نتایج و تحلیل عملکرد سیستم

۵-۱. نتایج کمی

آماره	مقدار	توضیحات
دقت تشخیص حرکت	~۹۲%	در روشنایی کافی
تاخیر	۵۰-۱۰۰ میلی ثانیه	از لحظه اتمام حرکت تا اجرای دستور
نرخ فریم AI	۲۵-۳۰ فریم در ثانیه	پردازش mediapipe
فاصله اسمی	۱۵۰-۲۰۰ سانتی متر	فاصله دوربین تا دست
بروزرسانی داده	۱ ثانیه	از ریزپردازنده تا سرور
همزمان سازی رابط کاربری	Real-Time	<۵۰ میلی ثانیه

پس از پیاده سازی کامل سیستم، عملکرد آن در شرایط مختلف مورد ارزیابی قرار گرفت. نتایج به دست آمده نشان می دهد که سیستم از نظر دقت و سرعت پاسخ دهی عملکرد قابل قبولی دارد.

- دقت تشخیص حرکات دست حدود ۹۲ درصد در شرایط نوری مناسب
- زمان تأخیر اجرای دستور بین ۵۰ تا ۱۰۰ میلی ثانیه
- نرخ پردازش تصویر بین ۲۵ تا ۳۰ فریم بر ثانیه
- فاصله مؤثر تشخیص دست بین ۱۵ تا ۲۰ سانتی متر از دوربین
- به روزرسانی داده های حسگر تقریباً هر ۱ ثانیه یکبار
- تأخیر همگام سازی رابط کاربری کمتر از ۵۰ میلی ثانیه

این اعداد نشان می دهند که سیستم برای کاربردهای عملی در کلاس درس کاملاً مناسب است و تجربه ای تقریباً بلادرنگ ارائه می دهد.

۵-۲. تحلیل نقاط قوت

از مهم‌ترین نقاط قوت سیستم می‌توان به موارد زیر اشاره کرد:

- استفاده هم‌زمان از هوش مصنوعی و IoT در یک معماری یکپارچه
- طراحی ماشین حالت برای جلوگیری از اجرای ناخواسته دستورات
- ارتباط بلادرنگ مبتنی بر WebSocket
- رابط کاربری مدرن با نمایش سه‌بعدی
- ساختار ماژولار و قابل توسعه

۵-۳. محدودیت‌ها

با وجود عملکرد مناسب سیستم، برخی محدودیت‌ها نیز مشاهده شد:

- وابستگی دقت تشخیص به شرایط نوری مناسب
- نیاز به پس‌زمینه نسبتاً ساده برای کاهش خطا
- پشتیبانی از تنها یک دست در هر زمان
- مصرف نسبتاً بالای CPU در پردازش ویدئو

این محدودیت‌ها می‌توانند در نسخه‌های آینده پروژه بهبود یابند.

۶. پیشنهادات توسعه آینده

برای ارتقای سیستم، پیشنهادهای زیر قابل بررسی است:

بهبود بخش هوش مصنوعی

- استفاده از مدل‌های عمیق‌تر مانند CNN-LSTM
- پشتیبانی از دو دست به صورت هم‌زمان
- بهینه‌سازی مدل با تکنیک Quantization

افزودن قابلیت‌های جدید

- کنترل صوتی (Speech Recognition)
- تشخیص چهره برای احراز هویت
- امکان تعریف حرکات سفارشی توسط کاربر

بهبود سخت‌افزار

- استفاده از ESP32 برای پردازش محلی
- افزودن سنسورهای بیشتر مانند PIR یا دود
- استفاده از رله و سروو برای کنترل واقعی تجهیزات

۷. جمع‌بندی نهایی

در این پروژه، یک سیستم کامل کلاس هوشمند طراحی و پیاده‌سازی شد که از مرحله جمع‌آوری داده‌های محیطی تا پردازش هوشمند و نمایش گرافیکی اطلاعات را پوشش می‌دهد. ترکیب فناوری‌های بینایی ماشین، ارتباط بلادرنگ، مدل‌سازی سه‌بعدی و اینترنت اشیا نشان می‌دهد که چگونه می‌توان با استفاده از ابزارهای مدرن مهندسی کامپیوتر، یک سیستم کاربردی و قابل توسعه ایجاد کرد.

این پروژه علاوه بر جنبه آموزشی، پتانسیل توسعه به یک محصول عملی در محیط‌های آموزشی، اداری و حتی خانگی را دارد.

- MediaPipe: [MediaPipe Solutions guide | Google AI Edge](#)
- FastAPI Documentation: [FastAPI](#)
- Three.js: [Three.js](#)
- Apache ECharts: [Apache ECharts](#)
- ESP8266 Documentation: [ESP8266 Arduino Core documentation](#)